Java w IntelliJ IDEA - gotowe klasy i praca z plikami

BLOK 7 - Math, Scanner, Random, String i pliki tekstowe

1. Dlaczego korzystamy z gotowych klas?

Nie każdą operację musimy programować od początku. Java zawiera standardową bibliotekę z wieloma klasami oferującymi gotowe metody. Na przykład `Math` umożliwia obliczenia matematyczne, `Scanner` pomaga odczytywać dane, a `Random` generuje wartości pseudolosowe. Wykorzystywanie standardowych narzędzi zmniejsza liczbę błędów i pozwala skoncentrować się na zadaniu.

Klasa jest typem zdefiniowanym w Javie. Niektóre metody wywołujemy bez tworzenia obiektu, np. `Math.sqrt(25)`, a w innych sytuacjach musimy utworzyć obiekt, np. `new Scanner(System.in)`. Rozróżnienie tego zapisu jest ważne: wywołanie klasycznej metody statycznej nie wymaga instancji, natomiast obiekt `Scanner` przechowuje informacje o tym, skąd czytamy dane.

2. Math - zaokrąglenia, pierwiastki i wartości bezwzględne

Klasa `Math` zawiera metody matematyczne. `Math.sqrt` oblicza pierwiastek kwadratowy, `Math.abs` wartość bezwzględną, a `Math.max` i `Math.min` wybierają większą lub mniejszą wartość. Metody te zwracają wynik; nie wypisują go automatycznie na ekranie.

```
System.out.println(Math.sqrt(49));
System.out.println(Math.abs(-8));
System.out.println(Math.max(15, 27));
System.out.println(Math.min(15, 27));
```

```
Wynik:
7.0
8
27
15
```

Wynik pierwiastka ma postać liczby zmiennoprzecinkowej, stąd `7.0`. Wartość bezwzględna `-8` wynosi 8, ponieważ chodzi o odległość liczby od zera. `Math.max` porównuje dwie wartości i zwraca większą. Jeśli napiszemy `double root = Math.sqrt(49);`, wynik zostanie zapisany w zmiennej, a nie wyświetlony.

3. Math.round, floor i ceil - trzy różne rodzaje zaokrąglenia

`Math.round` zaokrągla według swoich reguł do najbliższej liczby całkowitej, `Math.floor` zwraca największą liczbę całkowitą nieprzekraczającą argumentu, a `Math.ceil` najmniejszą liczbę całkowitą nie mniejszą niż argument. Ich zachowanie przy ujemnych liczbach często zaskakuje początkujących.

```
System.out.println(Math.round(2.7));
System.out.println(Math.floor(2.7));
System.out.println(Math.ceil(2.1));
System.out.println(Math.floor(-2.1));
```

```
Wynik:
3
2.0
3.0
-3.0
```

Przy liczbie -2.1 funkcja `floor` zwraca -3.0, ponieważ -3 jest największą liczbą całkowitą, która nie przekracza -2.1. Nie jest to zwykłe odcięcie cyfr po przecinku. Warto znać różnice między tymi metodami, zanim zastosujemy je np. do obliczania cen lub liczby potrzebnych paczek.

Metoda	**Co zwraca?**	**Pułapka**
`Math.sqrt(9)`	`3.0`	**wynik ma typ** `double`
`Math.floor(-1.2)`	`-2.0`	nie odcina ułamka w stronę zera
`random.nextInt(6)`	liczbę 0..5	6 jest wyłączone
`scanner.nextLine()`	cały wiersz tekstu	może odczytać pozostawiony koniec wiersza

4. Scanner - odczyt danych od użytkownika

Program konsolowy może prosić użytkownika o wprowadzenie informacji. Do ich odczytania służy m.in. klasa `Scanner`. Aby użyć jej bez pełnej nazwy pakietu, zapisujemy na początku pliku `import java.util.Scanner;`. Następnie tworzymy obiekt czytający ze standardowego wejścia `System.in`.

```
import java.util.Scanner;

Scanner scanner = new Scanner(System.in);
System.out.print("Podaj wiek: ");
int age = scanner.nextInt();
System.out.println("Za rok: " + (age + 1));
```

Jeżeli użytkownik wpisze 20, program wyświetli `Za rok: 21`. Metoda `nextInt()` oczekuje liczby całkowitej. Jeśli użytkownik wpisze tekst, którego nie można odczytać jako takiej liczby, może pojawić się `InputMismatchException`. Program nie domyśla się, jak zamienić dowolne słowo na wiek. W zastosowaniach praktycznych trzeba przewidzieć błędne dane wejściowe.

5. Różnica między nextInt i nextLine

`nextInt()` odczytuje liczbę, ale nie konsumuje automatycznie całego końca wiersza tak jak `nextLine()`. Dlatego bezpośrednio po wczytaniu liczby kolejna metoda `nextLine()` może odebrać pusty fragment pozostawiony w buforze. To częsty problem w zadaniach zawierających wiek i imię.

```
Scanner scanner = new Scanner(System.in);
System.out.print("Wiek: ");
int age = scanner.nextInt();
scanner.nextLine(); // odbiór końca wiersza
System.out.print("Imię: ");
String name = scanner.nextLine();
System.out.println(name + " ma " + age + " lat");
```

Gdy użytkownik poda `20` i następnie `Maja`, program prawidłowo otrzyma obie wartości. Dodatkowe `nextLine()` po `nextInt()` pobiera pozostałą część wiersza. Innym rozwiązaniem jest odczytywanie całych wierszy przez `nextLine()` i późniejsze parsowanie liczby metodą `Integer.parseInt`.

6. Random - liczby pseudolosowe i granice przedziałów

Klasa `Random` pozwala losować liczby pseudolosowe. Są one generowane algorytmicznie, a nie przez „magiczny” niezależny los przy każdym uruchomieniu. Szczególnie trzeba uważać na granice: `nextInt(bound)` zwraca liczbę od 0 włącznie do `bound` wyłącznie.

```
import java.util.Random;
Random random = new Random();
int dice = random.nextInt(6) + 1;
System.out.println(dice);
```

Wynik może być dowolną liczbą całkowitą od 1 do 6. `random.nextInt(6)` generuje 0, 1, 2, 3, 4 albo 5. Dopiero `+ 1` przesuwa cały przedział. Jeżeli chcemy uzyskać liczbę z zakresu od 10 do 15, możemy użyć `random.nextInt(6) + 10`. Taki wzór nie daje wartości 16.

7. Przewidywalne losowanie przez seed

Obiekt `Random` można utworzyć z określonym ziarnem, tzw. seed. Dla tego samego ziarna i tej samej kolejności wywołań otrzymamy tę samą sekwencję pseudolosową. To użyteczne podczas testowania programów: wyniki można odtworzyć zamiast analizować za każdym razem inne dane.

```
Random first = new Random(123);
Random second = new Random(123);
System.out.println(first.nextInt(100) == second.nextInt(100));
```

```
Wynik:
true
```

Oba obiekty startują z identycznym stanem generatora, dlatego ich pierwsze wyniki są takie same. Nie oznacza to, że `Random` jest odpowiedni do generowania haseł lub sekretów - do

zadań wymagających bezpieczeństwa służą inne narzędzia, np. `SecureRandom`.

8. String - przygotowanie danych tekstowych

Dane wpisane przez użytkownika mogą zawierać spacje, zbędne znaki lub różne wielkości liter. Metody `trim`, `toLowerCase`, `contains`, `startsWith` i `replace` pomagają takie informacje sprawdzać i przetwarzać. Przy każdej operacji pamiętaj, że `String` jest niemutowalny: metody zwracają nową wartość.

```
String input = "  Ala ma kota  ";
String clean = input.trim().toLowerCase();
System.out.println(clean);
System.out.println(clean.contains("kota"));
```

```
Wynik:
ala ma kota
true
```

`trim` usuwa białe znaki z początku i końca napisu w zakresie obsługiwanym przez tę metodę. `toLowerCase` zmienia litery na małe zgodnie z regułami lokalizacji. `contains("kota")` sprawdza, czy podany fragment występuje wewnątrz całego tekstu.

9. Parsowanie liczb odczytanych jako tekst

Gdy korzystamy z `nextLine()`, wszystkie wprowadzone dane otrzymujemy jako napisy. Aby wykonać działanie arytmetyczne, trzeba najpierw zamienić poprawnie sformatowany tekst na liczbę.

```
String entered = "42";
int number = Integer.parseInt(entered);
System.out.println(number * 2);
```

```
Wynik:
84
```

Gdy tekst nie jest prawidłowym zapisem liczby, `Integer.parseInt` zgłasza `NumberFormatException`. Nie wystarczy wpisać `int number = (int) entered;`, ponieważ `String` nie jest liczbą. Przy programach wczytujących dane można użyć obsługi wyjątków, którą poznasz w dalszym temacie.

10. Jak zapisać tekst do pliku?

Plik tekstowy pozwala zachować dane również po zakończeniu działania programu. Współczesna Java udostępnia klasę `Files` i typ `Path`, dzięki którym możemy zapisać tekst w prosty sposób. Dobrze wskazać kodowanie UTF-8, aby polskie litery były prawidłowo odtwarzane w różnych środowiskach.

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.charset.StandardCharsets;

Path path = Path.of("notatka.txt");
Files.writeString(path, "Przykładowy tekst", StandardCharsets.UTF_8);
```

Powyższy fragment zapisuje plik `notatka.txt` we wskazanym katalogu roboczym. Jeśli plik już istnieje, domyślny zapis przez `writeString` zastępuje jego zawartość. Operacja może się nie udać np. z powodu braku uprawnień. W metodzie `main` trzeba wtedy obsłużyć `IOException` lub zadeklarować `throws IOException`.

11. Odczyt pliku i rozumienie ścieżki

Odczytywanie pliku to odwrotność zapisywania. `Files.readString` zwraca tekst z istniejącego pliku. Przy ścieżce względnej, takiej jak `"notatka.txt"`, Java korzysta z aktualnego katalogu roboczego programu. To nie musi być katalog, w którym znajduje się plik źródłowy `.java`.

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.charset.StandardCharsets;

String content = Files.readString(
    Path.of("notatka.txt"), StandardCharsets.UTF_8
);
System.out.println(content);
```

Jeśli wcześniej zapisaliśmy `"Przykładowy tekst"`, właśnie taki napis zostanie wyświetlony. Jeżeli plik nie istnieje, odczyt zgłosi błąd związany z operacją wejścia/wyjścia. Zanim uznamy, że Java nie potrafi czytać plików, sprawdźmy rzeczywistą ścieżkę i katalog roboczy.

12. Try-with-resources i zamykanie zasobów

Niektóre sposoby obsługi plików wykorzystują strumienie, czytniki i zapisujące obiekty, które powinny zostać zamknięte. Konstrukcja `try-with-resources` pozwala zamknąć takie zasoby automatycznie, również gdy pojawi się wyjątek podczas pracy z plikiem.

```
import java.io.BufferedReader;
import java.nio.file.Files;
import java.nio.file.Path;

try (BufferedReader reader = Files.newBufferedReader(Path.of("notatka.txt"))) {
    System.out.println(reader.readLine());
}
```

Obiekt `reader` istnieje na czas pracy z plikiem i zostanie zamknięty po opuszczeniu bloku `try`. Przykład wymaga odpowiedniej obsługi `IOException`. Dla prostego odczytu małego pliku można skorzystać z `Files.readString`, ale przy większych plikach i przetwarzaniu wiersz po wierszu czytnik jest często wygodniejszy.

Przykład 1. Example01Math

Korzystamy z gotowych metod biblioteki matematycznej i odróżniamy wartości o typie `int` oraz `double`.

```
package block7;

public class Example01Math {
    public static void main(String[] args) {
        double x = -3.7;
        System.out.println(Math.abs(x));
        System.out.println(Math.pow(2, 10));
        System.out.println(Math.sqrt(81));
        System.out.println(Math.min(5, 9));
        System.out.println(Math.max(5, 9));
        System.out.println(Math.round(4.6));
        System.out.println(Math.PI);
    }
}
```

Wynik:

```
3.7
1024.0
9.0
5
9
5
3.141592653589793
```

- Zapis `double x = -3.7;` tworzy zmienną `x` typu `double` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println(Math.abs(x));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(Math.pow(2, 10));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(Math.sqrt(81));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(Math.min(5, 9));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(Math.max(5, 9));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 2. Example02Scanner

Program pobiera dane z klawiatury. Wynik zależy od tego, co wpisze użytkownik.

```
package block7;

import java.util.Scanner;

public class Example02Scanner {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Podaj imię: ");
        String name = scanner.nextLine();

        System.out.print("Podaj wiek: ");
        int age = Integer.parseInt(scanner.nextLine());

        System.out.println("Cześć " + name + ", masz " + age + " lat.");
    }
}
```

- W zapisie `Scanner scanner = new Scanner(System.in);` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.print("Podaj imię: ");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Zapis `String name = scanner.nextLine();` tworzy zmienną `name` typu `String` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.print("Podaj wiek: ");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Zapis `int age = Integer.parseInt(scanner.nextLine());` tworzy zmienną `age` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println("Cześć " + name + ", masz " + age + " lat.");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 3. Example03Random

Tworzymy generator pseudolosowy. Przy analizie pamiętajmy o granicy wyłącznej w `nextInt`.

```
package block7;

import java.util.Random;

public class Example03Random {
    public static void main(String[] args) {
        Random random = new Random();
        int die = random.nextInt(6) + 1;
        boolean coin = random.nextBoolean();

        System.out.println("Kostka: " + die);
        System.out.println("Moneta: " + (coin ? "orzeł" : "reszka"));
    }
}
```

- W zapisie `Random random = new Random();` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Zapis `int die = random.nextInt(6) + 1;` tworzy zmienną `die` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `boolean coin = random.nextBoolean();` tworzy zmienną `coin` typu `boolean` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println("Kostka: " + die);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println("Moneta: " + (coin ? "orzeł" : "reszka"));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 4. Example04StringParsing

Rozdzielamy jeden wiersz na pola i zamieniamy odpowiednie fragmenty tekstu na liczby.

```
package block7;
import java.util.Locale;
public class Example04StringParsing {
    public static void main(String[] args) {
        String record = "Anna;21;4.75";
        String[] parts = record.split(";");
        String name = parts[0];
        int age = Integer.parseInt(parts[1]);
        double average = Double.parseDouble(parts[2]);
        System.out.printf(Locale.US, "%s, wiek %d, średnia %.2f%n",
                name, age, average);
    }
}
```

Wynik:

```
Anna, wiek 21, średnia 4.75
```

- Zapis `String record = "Anna;21;4.75";` tworzy zmienną `record` typu `String` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `String[] parts = record.split(";");` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.
- Zapis `String name = parts[0];` tworzy zmienną `name` typu `String` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `int age = Integer.parseInt(parts[1]);` tworzy zmienną `age` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `double average = Double.parseDouble(parts[2]);` tworzy zmienną `average` typu `double` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

  • printf(Locale.US, ...) wypisuje liczbę z kropką dziesiętną niezależnie od ustawień systemu. Specyfikator %n kończy wiersz.

Przykład 5. Example05FileWrite

Wykonujemy zapis danych do pliku. Ważne jest ustalenie ścieżki i sposobu obsługi problemów z zapisem.

```
package block7;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class Example05FileWrite {
    public static void main(String[] args) throws IOException {
        Path path = Path.of("data", "output.txt");
        Files.createDirectories(path.getParent());
        Files.write(path, List.of("Java", "Pliki", "Tekst"));
        System.out.println("Zapisano: " + path.toAbsolutePath());
    }
}
```

- Instrukcja `System.out.println("Zapisano: " + path.toAbsolutePath());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 6. Example06FileRead

Odczytujemy plik utworzony w przykładzie 5. Najpierw uruchom Example05FileWrite, aby powstał plik data/output.txt.

```
package block7;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class Example06FileRead {
    public static void main(String[] args) throws IOException {
        Path path = Path.of("data", "output.txt");
        List<String> lines = Files.readAllLines(path);

        System.out.println("Liczba wierszy: " + lines.size());
        for (int i = 0; i < lines.size(); i++) {
            System.out.println("> " + lines.get(i));
        }
    }
}
```

- Instrukcja `System.out.println("Liczba wierszy: " + lines.size());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Nagłówek `for (int i = 0; i < lines.size(); i++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.

- Instrukcja `System.out.println("> " + lines.get(i));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Zadanie 7. InputProfile

Użyj Scanner do wczytania imienia i wieku. Wiek odczytaj jako tekst i zamień na int; wypisz czytelne zdanie z obiema wartościami.

Polecenie: Wypisz jedno czytelne podsumowanie danych.

```
package block7;

public class InputProfile {
    public static void main(String[] args) {
        // TODO
    }
}
```

Zadanie 8. DiceSimulation

Wylosuj 20 rzutów kostką (liczby 1-6) za pomocą Random. Wypisz wyniki i zlicz, ile razy wystąpiła każda liczba.

```
package block7;

public class DiceSimulation {
    public static void main(String[] args) {
        // TODO
    }
}
```

13. Zadania - klasy biblioteczne i pliki

Zadanie 1. Dla liczby -12 oblicz wartość bezwzględną. Dla 81 oblicz pierwiastek kwadratowy. Wyjaśnij, dlaczego pierwszy wynik może mieć typ `int`, a drugi `double`.

Zadanie 2. Napisz program pytający o imię i wiek. Użyj `Scanner`, a następnie wypisz komunikat „Za rok [imię] będzie mieć [wiek+1] lat”. Przetestuj odczyt danych przez `nextInt()` i `nextLine()`.

Zadanie 3. Wylosuj 20 wyników rzutu kostką. Sprawdź, czy każdy należy do zakresu 1..6. Zlicz wystąpienia każdej liczby.

Zadanie 4. Zapisz do pliku `wyniki.txt` imię i wynik punktowy. Odczytaj plik do napisu i wyświetl go w konsoli. Sprawdź, w jakim katalogu faktycznie powstał plik.

Zadanie 5. Zamień napis `" 123 "` na liczbę. Zastanów się, co trzeba zrobić ze spacjami i jaki wyjątek może pojawić się przy niepoprawnych danych.